



TÓPICOS AVANÇADOS EM ENGENHARIA DE SOFTWARE

Tecnologias do Observatório

Stack e decisões técnicas do ObiOne, Observatório Inteligente de Projetos para Consultorias

EQUIPE

Bruno Rocha
Cynthia Oliveira
Moisés Júnior
Raniel Silva

ORIENTADOR

Prof. Ivaldir Honório de Farias Júnior

INSTITUIÇÃO

Universidade de Pernambuco · Escola Politécnica de Pernambuco

LOCAL E DATA

Recife, 28 de maio de 2026

Este documento descreve as tecnologias que apoiam o desenvolvimento do ObiOne, o observatório inteligente de projetos para consultorias. As escolhas seguem três princípios: stack **pragmática** (sem abstrações desnecessárias), execução **local e reprodutível** (sem dependência de nuvem) e **baixo custo** de operação. Onde relevante, indica-se o requisito não funcional (RNF) atendido.

1. Backend

Núcleo da aplicação: API REST, persistência e regras de negócio.

Tecnologia	Versão	Papel
Python	3.11	Linguagem do backend
FastAPI	≥ 0.115	Framework web, rotas REST e injeção de dependências
Uvicorn	≥ 0.32	Servidor ASGI de execução
SQLAlchemy	2.0 (síncrono)	ORM e mapeamento objeto-relacional
psycopg	3.2	Driver de acesso ao PostgreSQL
Alembic	≥ 1.13	Migrações de banco versionadas
Pydantic	v2 (≥ 2.9)	Validação e serialização de dados (DTOs)
pydantic-settings	≥ 2.5	Configuração por ambiente, sem condicionais de produção no código
python-jose	≥ 3.3	Emissão e verificação de tokens JWT
passlib + bcrypt	1.7	Hash seguro de senhas
python-multipart	≥ 0.0.12	Upload de arquivos (multipart)
email-validator	≥ 2.2	Validação de endereços de e-mail

O SQLAlchemy é usado de forma **síncrona** porque o gargalo do sistema é a chamada ao LLM, não o banco. Isso simplifica o código sem perda prática de desempenho.

2. Banco de Dados

Tecnologia	Versão	Papel
PostgreSQL	16 (alpine)	Banco relacional, dados de projetos, extrações e comunidade

PostgreSQL atende a todo o modelo de dados (projetos, documentos, extrações, comentários, feedback). O controle de acesso semiaberto é aplicado na camada de serviço, por filtro de consulta, não por mecanismos do banco.

3. IA Generativa (Pipeline LLM)

Coração da contribuição: extração dos atributos do MPO e geração assistida de conteúdo.

Tecnologia	Versão	Papel
Ollama (Llama 3.1 8B)	local	Provider LLM padrão, execução local e zero custo de tokens
Instructor	≥ 1.6	Saída estruturada (JSON) validada por modelos Pydantic
OpenAI SDK	—	Acesso a provedores externos (OpenAI, Anthropic) pela porta plugável
python-docx	≥ 1.1	Leitura do texto dos documentos .docx de entrada
jsonschema	≥ 4.23	Validação da extração contra o schema versionado

O provider é selecionável por configuração (**mock**, **ollama**, **openai**), sem alterar o código. Cada extração registra a versão do prompt e o modelo usado, garantindo **reprodutibilidade** (RNF04). O uso local do Ollama como padrão mantém o **custo** controlado (RNF09).

4. Frontend

Interface do observatório, prototipada na plataforma Lovable.

Tecnologia	Versão	Papel
React	18	Biblioteca de interface
Vite	—	Build e servidor de desenvolvimento
TypeScript	—	Tipagem estática na interface
Tailwind CSS	—	Estilização utilitária
shadcn/ui	—	Biblioteca de componentes acessíveis (padrão Lovable)
Inter	—	Tipografia da interface

A interface segue uma identidade minimalista e tecnológica, inspirada em plataformas modernas de analytics e observabilidade, com foco em escaneabilidade e hierarquia visual clara (RNF02, usabilidade).

5. Arquitetura e Padrões

A organização do código favorece a **manutenibilidade** (RNF03).

Padrão	Papel
Clean architecture pragmática	Cada contexto separa models , repository , service , schemas , router , dependencies e exceptions
Bounded contexts	auth , projects , documents , extractions , comments , resumos , drafts , likert , feed , exports e os novos themes , portfolio , synthesis
Ports & adapters	Integrações externas plugáveis: LLM (mock, Ollama, Instructor) e armazenamento de arquivos (filesystem)

Padrão	Papel
Unit of Work	Serviços manipulam uma unidade de trabalho transacional, não a sessão do banco diretamente

A camada de serviço é livre de framework (sem importações de FastAPI), o que mantém as regras de negócio testáveis e independentes da web.

6. Qualidade e Testes

Tecnologia	Versão	Papel
pytest	≥ 8.3	Testes em três níveis: unitário, integração e ponta a ponta
httpx (TestClient)	≥ 0.27	Testes ponta a ponta da API
pytest-cov	≥ 5.0	Medição de cobertura de testes
ruff	≥ 0.7	Lint e formatação (regras E, F, W, I, N, UP, B, C4, SIM)

Os testes se dividem em **unitários** (lógica pura, sem I/O), **integração** (PostgreSQL real, transação com rollback) e **ponta a ponta** (fluxo HTTP completo via TestClient).

7. Infraestrutura e DevOps

Tecnologia	Papel
Docker + Compose	Ambiente local reproduzível (PostgreSQL + backend) (RNF06)
GitHub Actions	Integração contínua: ruff + pytest contra um serviço PostgreSQL 16
Alembic	Migrações de banco, geradas e aplicadas de forma versionada
Make	Atalhos de desenvolvimento (up , test , migrate , lint , seed)
Git / GitHub	Versionamento e fluxo de trabalho por Pull Request

Síntese

O stack é deliberadamente conservador e local-first: FastAPI e PostgreSQL no backend, Ollama com Instructor para a IA Generativa, React com Vite no frontend, e Docker com GitHub Actions para ambiente e integração contínua. As decisões priorizam reprodutibilidade científica, baixo custo e manutenibilidade, em linha com os requisitos não funcionais do projeto.